Lab 3 Report - 11/12/26

CSEN 383

Zachary Common and Calianna Collins

https://github.com/Eval427/xv6-riscv

https://youtu.be/jObM2_cCV6c

Modified Files

```
\ = Modified
+ = Added
- = Removed

\ Makefile

\ kernel/defs.h
\ kernel/proc.c
\ kernel/proc.h
\ kernel/syscall.c
\ kernel/syscall.h
\ kernel/sysproc.c
\ kernel/trampoline.S
\ kernel/trap.c
+ kernel/thread.c
+ kernel/thread.h

+ user/lab3test.c
+ user/thread.c
+ user/thread.h
\ user/user.h
\ user/usys.pl
```

Part 1

In part 1, we added one new syscall: `clone(void*)`.

Process Changes

In order for threads to be unique processes, we had to edit the PCB in order to contain a thread id value, labeled `tid`. Processes also need to store the number of threads they currently own, since they have an upper limit of 20.

```
// Per-process state
struct proc {
```

```
  struct spinlock lock;

  // p->lock must be held when using these:
  enum procstate state;        // Process state
  void *chan;                  // If non-zero, sleeping on chan
  int killed;                  // If non-zero, have been killed
  int xstate;                  // Exit status to be returned to parent's
wait
  int pid;                     // Process ID
  int tid; /*<--*/             // Thread ID
  int threads; /*<--*/         // Number of threads in a process

  // wait_lock must be held when using this:
  struct proc *parent;         // Parent process

  // these are private to the process, so p->lock need not be held.
  uint64 kstack;               // Virtual address of kernel stack
  uint64 sz;                   // Size of process memory (bytes)
  pagetable_t pagetable;       // User page table
  struct trapframe *trapframe; // data page for trampoline.S
  struct context context;      // swtch() here to run process
  struct file *ofile[NOFILE];  // Open files
  struct inode *cwd;           // Current directory
  char name[16];               // Process name (debugging)
  int syscall_count;           // Total number of system calls that the
current process has made
  int tickets;                 // Number of tickets for the lottery
scheduling process
  int ticks;                   // Number of times this process was called
(chosen) by scheduler
  int pass;
  int stride;
};
```

`trampoline.S` was also changed to a new version provided in the assignment, which also for multi-threaded processes.

Threads vs Children

To create `clone()`, we copied and modified `fork()` so that it used the parent's address space instead of a new one.

```
int clone(void *stack) {
  struct proc *thread;
  struct proc *parent = myproc();
  int tid;
  int pid;
  int i;

  if (stack == 0) return -1;
```

```
  if (parent->tid != 0) return -1; // Threads cannot create threads

  if ((thread = allocthread(parent)) == 0) {
    return -1;
  }

  tid = thread->tid;
  thread->pagetable = parent->pagetable;
  thread->sz = parent->sz;

  if(mappages(thread->pagetable,
              TRAPFRAME - PGSIZE * tid,
              PGSIZE,
              (uint64)thread->trapframe,
              PTE_R | PTE_W) < 0){
    thread->pagetable = 0;    // Prevent freeproc() from freeing parent
table
    freeproc(thread);
    release(&thread->lock);
    return -1;
  }

  *(thread->trapframe) = *(parent->trapframe);

  thread->trapframe->a0 = 0;
  thread->trapframe->sp = (uint64)stack;

    // increment reference counts on open file descriptors.
  for(i = 0; i < NOFILE; i++)
    if(parent->ofile[i])
      thread->ofile[i] = filedup(parent->ofile[i]);
  thread->cwd = idup(parent->cwd);

  safestrcpy(thread->name, parent->name, sizeof(thread->name));

  pid = thread->pid;

  release(&thread->lock);

  acquire(&wait_lock);
  thread->parent = parent;
  release(&wait_lock);

  acquire(&thread->lock);
  thread->state = RUNNABLE;
  release(&thread->lock);

  return pid;
}
```

Each thread also needed a new way to allocate it, so we created a new function `allocthread(struct proc *parent)`, which takes in a parent proc struct and assigns the thread's characteristics based on this

parent struct.

```
static struct proc* allocthread(struct proc *parent)
{
  struct proc *p;
  uint64 K = 10000;

  for(p = proc; p < &proc[NPROC]; p++) {
    acquire(&p->lock);
    if(p->state == UNUSED) {
      goto found;
    } else {
      release(&p->lock);
    }
  }
  return 0;

found:
  if (parent->threads >= 20) {
    release(&p->lock);
    return 0;
  }

  p->pid = allocpid();
  p->tid = ++parent->threads;
  p->state = USED;
  p->tickets = 100; // Need at least one ticket to run at all - Give 100
for faster testing
  p->syscall_count = 0;
  p->stride = K / p->tickets;
  p->pass = p->stride;
  p->ticks = 0;

  p->pagetable = 0;
  p->sz = 0;

  p->trapframe = (struct trapframe *) kalloc();
  if (p->trapframe == 0) {
    freeproc(p);
    release(&p->lock);
    return 0;
  }

  memset(&p->context, 0, sizeof(p->context));
  p->context.ra = (uint64)forkret;
  p->context.sp = p->kstack + PGSIZE;

  return p;
}
```

This struct also calls `freeproc(struct proc*)`, which we edited so that threads are freed differently from other processes. Notably, they have to keep the trapframe of the parent process intact, instead of freeing it back to the kernel.

```
static void
freeproc(struct proc *p)
{
  if (p->tid > 0) {
    uvmunmap(p->pagetable, TRAPFRAME - PGSIZE * p->tid, 1, 0);
  }

  if(p->trapframe)
    kfree((void*)p->trapframe);
  p->trapframe = 0;
  if(p->tid == 0 && p->pagetable)
    proc_freepagetable(p->pagetable, p->sz);
  p->pagetable = 0;
  p->sz = 0;
  p->pid = 0;
  p->parent = 0;
  p->name[0] = 0;
  p->chan = 0;
  p->killed = 0;
  p->xstate = 0;
  p->state = UNUSED;
}
```

This allowed us to avoid making any changes to `wait()`, since all the changes to wait are actually relevant to processes freeing.

```
// kernel/defs.h:116
int clone(void *stack); // clone
```

```
// kernel/syscall.h:28
#define SYS_clone    27 // clone
```

```
// kernel/syscall.c: 116 & 147
extern uint64 sys_clone(void);
...
[SYS_clone] sys_clone, // clone: syscall entry
```

```
// kernel/sysproc.c
uint64 sys_clone(void) {
```

```
  uint64 stack;
  argaddr(0, &stack);

  return clone((void *)stack);
}
```

```
// user/user.h:31
int clone(void*); // clone
```

```
# user/usys.pl:44
entry("clone"); # clone syscall
```

Part 2

Part 2 implements a user-level thread library for creating multi-threaded programs. It creates two new files, `thread.c` and `thread.h`, which serve as library files for other programs to be able to use the new clone syscall correctly.

Thread Library

In addition to making the files for the threads, we had to edit the `Makefile` to include the new library in the compilation.

```
ULIB = $U/ulib.o $U/usys.o $U/printf.o $U/umalloc.o $U/thread.o
```

`thread.h` defines a new struct lock_t, which controls the processing of threads in the kernel. It is a simple boolean value which defines whether or not a thread is in use.

```
typedef struct lock_t {
  uint locked;
} lock_t;

int thread_create(void *(start_routine)(void*), void *arg);
void lock_init(struct lock_t* lock);
void lock_acquire(struct lock_t* lock);
void lock_release(struct lock_t* lock);
```

thread.c

```
// user/thread.c
#include "kernel/types.h"
#include "kernel/riscv.h"
```

```c
#include "user/user.h"
#include "user/thread.h"

int thread_create(void *(start_routine)(void*), void *arg) {
  int pid;
  char *stack = malloc(PGSIZE);
  if (stack == 0) {
    printf("thread_create failed\n");
    exit(1);
  }

  pid = clone(stack + PGSIZE);
  if (pid < 0){
    free(stack);
    printf("thread_create failed\n");
    exit(1);
  }
  else if (pid == 0){
    start_routine(arg); // child executes specified function
    exit(0);
  }

  return 0;
}

void lock_init(struct lock_t* lock) {
  lock->locked = 0;
}

void lock_acquire(struct lock_t* lock) {
  while (__sync_lock_test_and_set(&lock->locked, 1) != 0)
    ;

  __sync_synchronize();
}

void lock_release(struct lock_t* lock) {
  __sync_synchronize();

  __sync_lock_release(&lock->locked);
}
```

`thread.c` primarily defines `thread_create(void *(start_routine)(void*), void *arg)`, which allows threads to be created in user programs. It uses malloc to allocate new stack space inside a process, and sends that added to PGSIZE to clone. Once the thread is created, it gives it a user defined process to execute. Each of the lock functions takes advantage of GCCs built in operations to create atomic_test_and_set checks which allow locks to be controlled. These locks control a single boolean lock value which is synchronized using another GCC operation.

Output

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ lab3test 10 3
Round 1: thread 0 is passing the token to thread 1
Round 2: thread 1 is passing the token to thread 2
Round 3: thread 2 is passing the token to thread 0
Round 4: thread 0 is passing the token to thread 1
Round 5: thread 1 is passing the token to thread 2
Round 6: thread 2 is passing the token to thread 0
Round 7: thread 0 is passing the token to thread 1
Round 8: thread 1 is passing the token to thread 2
Round 9: thread 2 is passing the token to thread 0
Round 10: thread 0 is passing the token to thread 1
Frisbee simulation has finished, 10 rounds played in total
$ lab3test 21 20
Round 1: thread 0 is passing the token to thread 1
Round 2: thread 1 is passing the token to thread 2
Round 3: thread 2 is passing the token to thread 3
Round 4: thread 3 is passing the token to thread 4
Round 5: thread 4 is passing the token to thread 5
Round 6: thread 5 is passing the token to thread 6
Round 7: thread 6 is passing the token to thread 7
Round 8: thread 7 is passing the token to thread 8
Round 9: thread 8 is passing the token to thread 9
Round 10: thread 9 is passing the token to thread 10
Round 11: thread 10 is passing the token to thread 11
Round 12: thread 11 is passing the token to thread 12
Round 13: thread 12 is passing the token to thread 13
Round 14: thread 13 is passing the token to thread 14
Round 15: thread 14 is passing the token to thread 15
Round 16: thread 15 is passing the token to thread 16
Round 17: thread 16 is passing the token to thread 17
Round 18: thread 17 is passing the token to thread 18
Round 19: thread 18 is passing the token to thread 19
Round 20: thread 19 is passing the token to thread 0
Round 21: thread 0 is passing the token to thread 1
Frisbee simulation has finished, 21 rounds played in total
```

Contributions

Both Zack and Calianna worked on the code together in person with extra debugging by Zack to work out a few remaining issues. Calianna wrote the bulk of the lab report while Zack reviewed and edited it.